CI/CD Integration with GitHub Actions

LESSON SPECIFICATIONS & ROADMAP

• Track: Playwright Automation with JavaScript + AI • Level: Advanced
• Estimated Effort: 2.5–3 Hours • Prerequisites: Days 1–14 concepts (Git, test runner, config, reporting, and cross-browser
• Runtime: Pure Modern JavaScript (.js) • Runner: @playwright/test

Automate test execution on every pull request using GitHub Actions. Build reliable CI pipelines with official Playwright actions, publish downloadable HTML test reports, and diagnose cloud build failures.

Day 15 Learning Objectives

'Understand CI/CD core concepts: Continuous Integration, automated pull request gating, and rapid regression feedback.
'Author production GitHub Actions workflow YAML files: triggers, runners, jobs, steps, and environment variables.
'Build an enterprise Playwright CI pipeline: Node.js setup, npm ci, browser installation with --with-deps, and test execution.
'Configure artifact uploads to preserve Playwright HTML reports and Trace Viewer archives upon build failure.
'Diagnose and troubleshoot common CI build failures: missing dependencies, uninstalled browsers, and environment misconfigurations.
'Expose test failures clearly in pull request checks without masking defects through unconditional retries.

Automated tests have zero business value if they only run on a QA engineer's local laptop. Automated quality gates must run automatically in the cloud on every proposed code change.

What Is Continuous Integration (CI)?

Continuous Integration is the software development practice where developers merge their code changes into a central repository frequently. Each merge triggers an automated build and test pipeline to detect integration errors as early as possible.

The CI Feedback Loop

1. Developer creates a feature branch and pushes code to GitHub.
2. A Pull Request (PR) is opened.
3. The CI runner (e.g. GitHub Actions) spins up a virtual Linux machine (`ubuntu-latest`).
4. The pipeline installs dependencies and runs the Playwright test suite in headless mode.
5. If any test fails, the PR is blocked from merging into `main`.
6. Developers receive immediate failure reports in under 5 minutes!

Why Manual QA Gates Slow Down Delivery

Waiting for manual testing before every deployment creates multi-day bottlenecks. CI pipelines with automated Playwright tests allow engineering teams to deploy multiple times per day with high confidence.

GitHub PR Check Status (A pull request waiting for automated tests to pass.)

```
[Checks] Playwright E2E Tests / test (pull_request) — In progress...
'  45 tests passed
'  1 test failed (TC-AUTH-02: invalid login)
>> Merge blocked until checks pass
```

GitHub Actions is the industry standard CI/CD platform built directly into GitHub repositories.

Key Components of a Workflow File

Workflows are defined in YAML format inside the `.github/workflows/` directory:

1. **Triggers (`on`):** Defines what events trigger the pipeline (e.g. `push` to main, `pull_request`, or scheduled cron).
2. **Jobs (`jobs`):** A collection of steps that execute on a virtual runner. Jobs run in parallel by default.
3. **Runners (`runs-on`):** The operating system hosting the job (e.g. `ubuntu-latest`, `windows-latest`, `macos-latest`).
4. **Steps (`steps`):** Linear tasks that execute commands (`run`) or reuse pre-built actions (`uses`).
5. **Secrets & Env (`env`):** Secure values injected from GitHub Secrets (e.g. `${{ secrets.TEST_USER_PASSWORD }}`).

Basic Workflow Triggers (Triggering on push and pull requests targeting main.)

```yaml
name: Playwright Tests
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
```

Here is the official, production-ready Playwright GitHub Actions workflow:

```yaml
# .github/workflows/playwright.yml
name: Playwright Regression Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    timeout-minutes: 60
    runs-on: ubuntu-latest

    steps:
      # Step 1: Check out the git repository
      - name: Checkout Code
        uses: actions/checkout@v4

      # Step 2: Set up Node.js LTS
      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      # Step 3: Clean install project dependencies (npm ci)
      - name: Install Project Dependencies
        run: npm ci

      # Step 4: Install Playwright browser binaries with Linux OS dependencies
      - name: Install Playwright Browsers
        run: npx playwright install --with-deps

      # Step 5: Execute Playwright test suite in headless mode
      - name: Run Playwright Tests
        run: npx playwright test
        env:
          CI: true
          BASE_URL: https://staging.rootaccess.org
          TEST_STUDENT_EMAIL: ${{ secrets.TEST_STUDENT_EMAIL }}
          TEST_STUDENT_PASSWORD: ${{ secrets.TEST_STUDENT_PASSWORD }}

      # Step 6: Upload HTML Report & Trace Artifacts (always runs even if tests fail!)
      - name: Upload Test Report & Traces
        uses: actions/upload-artifact@v4
        if: always()
        with:
          name: playwright-report
          path: playwright-report/
          retention-days: 14
```

Critical Pipeline Nuance: `--with-deps`
In a headless Linux runner, browsers require OS-level graphics and font libraries (like libx11, libgstreamer). Running `npx playwright install --with-deps` automatically installs these system dependencies using `apt-get`, preventing "Browser closed unexpectedly" startup errors!

Handling Artifact Upload on Failure (Ensuring test reports are accessible when a build breaks.)

```yaml
- name: Upload Playwright Report
  uses: actions/upload-artifact@v4
  if: always() # CRITICAL: runs on both success and failure!
  with:
    name: playwright-report
    path: playwright-report/
    retention-days: 30
```

When a CI build fails, engineers must systematically diagnose whether the cause is a **pipeline/environment defect** or a **software regression**.

3 Most Common CI Pipeline Errors

1. **Missing `--with-deps`:** Error: `browserType.launch: Host system is missing dependencies to run browsers`.
   - **Fix:** Update command to `npx playwright install --with-deps`.
2. **Missing Dependencies (`npm install` omitted):** Error: `Cannot find module '@playwright/test'`.
   - **Fix:** Add `run: npm ci` step prior to running tests.
3. **Incorrect Artifact Path:** Error: `Path does not exist: playwright-report/`.
   - **Fix:** Verify the `outputFolder` in `playwright.config.js` matches the artifact upload path.

Distinguishing Test Failures from Infrastructure Failures

- **Infrastructure Failure:** Step 2, 3, or 4 fails during npm install or browser download. The tests never ran.
- **Test Failure:** Step 5 executes, runs 20 tests, and fails with `1 failed`. Download the artifact from the summary page to view the trace.

Inspecting GitHub Actions Summary (Finding uploaded artifacts on GitHub.)

```
1. Open GitHub repository -> "Actions" tab.
2. Click the failed workflow run.
3. Scroll to the bottom "Artifacts" section.
4. Download "playwright-report.zip".
5. Unzip and view index.html with embedded traces!
```

requirements.

Assignment Starter Code (Day 15)

```
# .github/workflows/playwright_pipeline.yml
# Production-ready GitHub Actions CI/CD Pipeline for Playwright Automation

name: Playwright Regression Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  e2e-testing:
    name: Playwright Test Suite
    timeout-minutes: 30
    runs-on: ubuntu-latest

    steps:
      # 1. Check out repository code
      - name: Checkout Code Repository
        uses: actions/checkout@v4
```

DAY 15 INTERVIEW PREPARATION QUESTION BANK

Q1. How do you design a GitHub Actions pipeline for Playwright, and what steps are mandatory?

[Difficulty: Intermediate] Evaluation Focus: CI/CD pipeline architecture, official actions, and execution sequence.

Model Answer: A production Playwright GitHub Actions pipeline runs on `ubuntu-latest` and consists of 6 sequential steps: 1) `actions/checkout@v4` to pull repository code; 2) `actions/setup-node@v4` with Node 20 and npm caching; 3) `npm ci` for clean, deterministic dependency installation; 4) `npx playwright install --with-deps` to download browser binaries and Linux OS packages; 5) `npx playwright test` with environment variables injected from GitHub Secrets; and 6) `actions/upload-artifact@v4` with `if: always()` to archive the HTML test report and traces for 14 to 30 days regardless of whether tests passed or failed.

Follow-Up Question: "Why is npm ci preferred over npm install in CI pipelines?"

Q2. Why is the --with-deps flag required when installing Playwright browsers on Linux CI runners?

[Difficulty: Intermediate] Evaluation Focus: Linux headless browser architecture and container dependencies.

Model Answer: Clean Linux virtual machine runners (like `ubuntu-latest`) do not include desktop graphical libraries by default. Headless Chromium, Firefox, and WebKit binaries require system packages for font rendering, audio processing, and window management (such as libx11, libgbm, and libasound). The `--with-deps` flag tells Playwright to automatically invoke the operating system package manager (`apt-get`) to install these prerequisite Linux libraries, preventing runtime browser launch crashes.

Follow-Up Question: "Can you use official Microsoft Playwright Docker containers instead of installing dependencies manually?"

Q3. How do you investigate a Playwright test failure that only occurs on GitHub Actions but passes locally?

[Difficulty: Advanced] Evaluation Focus: Environmental discrepancy triage and Trace Viewer debugging.

Model Answer: Discrepancies between local and CI runs are usually caused by: 1) Timing and network latency differences on resource-constrained virtual machines; 2) Screen resolution/viewport differences (defaulting to 1280x720 in headless); 3) Missing environment variables or unseeded database state; or 4) Operating system differences (Linux vs macOS/Windows). I investigate by downloading the `playwright-report` artifact from the GitHub Actions run summary and opening the embedded `trace.zip` in Trace Viewer. This lets me inspect the exact network waterfall, DOM state, and console errors from the CI runner.

Follow-Up Question: "How can you simulate CI environment constraints on your local machine?"

Q4. What is the purpose of if: always() on the artifact upload step in a GitHub Actions workflow?

[Difficulty: Beginner] Evaluation Focus: Workflow condition logic and failure artifact preservation.

Model Answer: By default, GitHub Actions halts execution immediately if any step fails (exits with a non-zero code). When a Playwright test fails, the test step returns exit code 1. Without `if: always()`, the artifact upload step would be skipped, and the HTML report and failure traces would be lost. Specifying `if: always()` forces GitHub Actions to execute the upload step whether the tests passed, failed, or timed out, guaranteeing that engineers have diagnostic evidence to inspect.

Follow-Up Question: "How does if: failure() differ from if: always()?"

Q5. How do you securely pass credentials and API keys into a GitHub Actions Playwright run?

[Difficulty: Intermediate] Evaluation Focus: Secret management and environment variable injection.

Model Answer: We store credentials as Encrypted Secrets in the repository settings (e.g. `TEST_USER_PASSWORD`). In the workflow YAML file, under the test execution step, we map secrets into environment variables using the `${{ secrets.NAME }}` syntax. Playwright tests read these at runtime via `process.env.TEST_USER_PASSWORD`. GitHub automatically masks secret values in console logs, preventing credentials from appearing in public build transcripts.

Follow-Up Question: "What precautions must be taken with Pull Requests submitted from public repository forks regarding secrets?"

1. Where must GitHub Actions workflow files be saved in your Git repository?

Key Concept: GitHub Actions automatically parses any YAML files stored inside the .github/workflows/ directory in the repository root.

2. Why should npm ci be used instead of npm install in CI workflows?

Key Concept: npm ci ensures clean, deterministic, and faster installations by strictly adhering to package-lock.json without modifying it.

3. What does the --with-deps flag do during npx playwright install on Linux runners?

Key Concept: --with-deps instructs Playwright to install system-level Linux dependencies (like libx11 and libgbm) required by browser binaries.

4. Which step condition ensures that test reports are uploaded even when tests fail in GitHub Actions?

Key Concept: if: always() forces the step to execute regardless of the failure status of previous steps, ensuring failure reports are captured.

5. How do you reference an encrypted repository secret named TEST_PASSWORD in a GitHub Actions workflow step?

Key Concept: GitHub Actions expressions use the ${{ secrets.SECRET_NAME }} syntax to inject encrypted repository secrets into environment variables.

6. What is the default virtual runner operating system recommended for cost-effective, fast Playwright CI execution?

Key Concept: ubuntu-latest is the fastest, most cost-effective virtual runner on GitHub Actions and supports all Playwright browser engines.

7. How can developers inspect the HTML test report generated by a failed GitHub Actions run?

Key Concept: Uploaded artifacts appear as downloadable zip files at the bottom of the GitHub Actions run summary page.

8. What happens to a Pull Request if an automated Playwright test step exits with code 1 in GitHub Actions?

Key Concept: An exit code of 1 marks the check as failed, acting as a quality gate that prevents merging broken code into the protected main branch.

9. Which official action from GitHub is used to check out repository code into the CI runner workspace?

Key Concept: actions/checkout@v4 is the standard official GitHub action used to check out repository source code so subsequent steps can access it.

10. Why should CI pipelines avoid setting retries to high numbers (e.g. retries: 5)?

Key Concept: Setting excessive retries masks underlying timing and synchronization bugs, degrades test trust, and artificially prolongs CI pipeline durations.